In [1]:
```
# creating a sparksession
from pyspark.sql import SparkSession
import getpass
username = getpass.getuser()
spark = SparkSession. \
builder. \
config('spark.ui.port', '0'). \
config("spark.sql.warehouse.dir", f"/user/{username}/warehouse"). \
enableHiveSupport(). \
master('yarn'). \
getOrCreate()
```

In [2]:
```
# creating a local list
words = ("big","Data","Is","SUPER","Interesting","BIG","data","IS","A","Trendi
```

In [3]:
```
# we will parallelize it so that we can create an RDD
# let's say sparkContext.textfile and we create for example named:  base_rdd
# base_rdd = loading data from hdfs (1 gb file)



How to check the no.of RDD partitions?

Let's consider an example file of 1GB(1024MB) in HDFS

No.of blocks in HDFS for this 1GB file (with default block size of 128MB)

1024MB / 128MB = 8 Blocks

so there will be 8 blocks in your hdfs( which is simply partitions in spark)
when this will be loaded to the memory   then there will be
8 partitions in your rdd...

This indicates no.of RDD partitions will be 8 Partitions (Because no.of RDDpar
```

In [ ]:
```
words_rdd = spark.sparkContext.parallelize(words)
```

above data is very small in KB's # what will be the no of partitions created in RDD for this # RDD is made up of partitions as it is distributed

In [ ]:
```
# to check the no of partitions in rdd, use the function getNumPartitions() to
```

words_rdd.getNumPartitions() # so this words_rdd has 2 partitions here # here we are having around 3KB data , but how will be there 2 partitions present # when u parallelize the data the no of partitions i defined by a property spark.sparkContext.defaultParallelism # spark.sparkContext.defaultParallelism will define how many partitions should present when we parallelize -- default size which is 2 here # so we are getting the partitions when we parallelize # even the data is small by default the partitions will be 2 only

In [5]:
```
spark.sparkContext.defaultParallelism
```

Out[5]: 2

u have an orders file in orders folder which is around the size 3mb # in local gateway node , check as hadoop fs -ls -h filelocation

In [6]:
```
orders_rdd = spark.sparkContext.textFile("/public/trendytech/retail_db/orders/
# we loaded here and try to check the no of partitions here
```

In [7]:
```
orders_rdd.getNumPartitions()
```

Out[7]: 2

In [8]:
```
spark.sparkContext.defaultMinPartitions
```

Out[8]: 2

The file size in the above example is less than 128MB . => There wouldbe only one block in HDFS => no.of RDD partitions should be 1. no.of RDD partitions = no.of blocks in HDFS getNumPartitions() However, on executing the getNumPartitions() function, it is resulting in 2 partitions. This is because of the property defaultMinPartitions set to 2 (i.e., if the number of RDD partitions is < 2, consider the default value. If > 2 then consider the evaluated value based on number of blocks) defaultMinPartitions spark.SparkContext.defaultMinPartitions defaultParallelism spark.SparkContext.defaultParallelism is a property that indicates the no.of default tasks that can run in parallel. even if the file size is less than 128mb , still we have 2 partitions as defaultMinPartitions is 2 100 mbfile = 2 partitions 150 mbfile = 2 partitions 300 mbfile = 3partitions# lets take some big file here which is around 350 mb # there would be 3 blocks in HDFS . so there will be 3 partitions in rdd.

In [9]:
```
base_rdd = spark.sparkContext.textFile("/public/trendytech/bigLog.txt")
```

In [10]:
```
base_rdd.getNumPartitions()
```

Out[10]: 3

In [11]:
```
orders_rdd = spark.sparkContext.textFile("/public/trendytech/retail_db/orders/
```

In [12]:
```
mapped_rdd = orders_rdd.map(lambda x:(x.split(",")[3],1))
```

In [13]:
```
results = mapped_rdd.reduceByKey(lambda x,y:x+y)
```

In [14]:
```
results.collect()
```

Out[14]:
```
[('CLOSED', 7556),
 ('CANCELED', 1428),
 ('PENDING_PAYMENT', 15030),
 ('COMPLETE', 22899),
 ('PROCESSING', 8275),
 ('PAYMENT_REVIEW', 729),
 ('PENDING', 7610),
 ('ON_HOLD', 3798),
 ('SUSPECTED_FRAUD', 1558)]
```

Easy way to get the frequency of the words using countByValue

In [15]:
```
orders_rdd = spark.sparkContext.textFile("/public/trendytech/retail_db/orders/
# instead of two times mapping and then reduceby key:  we can use countByValue
```

In [16]:
```
mapped_rdd = orders_rdd.map(lambda x:(x.split(",")[3]))
# here we are simply getting the words only
```

In [ ]:
```
# but here is some difference present
1. reduceByKey is a transformation here whereas
2. countByValue is an action  (on this result,we cannot further do parallelism

we will prefer countByValue when we treat it as final output and no further op

if u want to perform some operation again on the result :
[('CLOSED', 7556),
 ('CANCELED', 1428),
 ('PENDING_PAYMENT', 15030),
 ('COMPLETE', 22899),
 ('PROCESSING', 8275),
 ('PAYMENT_REVIEW', 729),
 ('PENDING', 7610),
 ('ON_HOLD', 3798),
 ('SUSPECTED_FRAUD', 1558)]


then countByValue should not be used.

when further operations (transformations) need to be done here ,

we will use still the reduceByKey as results are distributed  across the clust
```

In [ ]:
```
countByValue- Is an action that gives the same output as map and
reduceByKey combined.
countByValue = map + reduceByKey

Seems that countByValue is a better choice as it requires less lines of code t
achieve the same results.
However, countByValue cannot replace map & reduceByKey always because--

    Note:--
countByValue is an action where output is captured on a local / gateway
node and not on the cluster (implies, no further processing in parallel
can happen on the results from countByValue)

reduceByKey on the other hand is just a transformation (implies, there
can be further parallel processing on the results from reduceByKey)
If you need to do further parallel processing, then use
map+reduceByKey


If the resultant output would be the final output where no further
processing in parallel is required, then countByValue would be the right
choice.
```

In [17]:
```
mapped_rdd.countByValue()
# now all results will be stored in local
# shorter code
```

```
# here couuntByValue is an action here
# map+reducebykey is nothing but the countByValue
```

Out[17]:
```
defaultdict(int,
            {'CLOSED': 7556,
             'PENDING_PAYMENT': 15030,
             'COMPLETE': 22899,
             'PROCESSING': 8275,
             'PAYMENT_REVIEW': 729,
             'PENDING': 7610,
             'ON_HOLD': 3798,
             'CANCELED': 1428,
             'SUSPECTED_FRAUD': 1558})
```

In [ ]: